

EKF Measurement Models

Hardware Porting Guide

MPU6050 IMU, RPLiDAR, wheel encoders

Package: ekf_estimator | Target platform: ROS 2 Jazzy

Generated 2026-05-13

1. Overview of changes

The EKF in this workspace fuses three sensor inputs into a 5-state estimate $x = [x, y, \theta, v, \omega]^T$. Before this round of edits the ROS 2 wrapper was already wired for hardware-standard message types (sensor_msgs/Imu, sensor_msgs/LaserScan, nav_msgs/Odometry), but the LiDAR path was a stub (no scan-matching) and a few sim-only behaviours leaked into the runtime. This document records what changed, why, and how to deploy the same node on a real diff-drive robot equipped with an MPU6050 IMU, an RPLiDAR, and wheel encoders.

1.1 Files touched

File	Change
src/ekf_estimator/include/ekf_estimator/ekf_node.hpp	Added ICP helper declarations, gyro-bias / scan-matching parameters, and scan-pose accumulator state.
src/ekf_estimator/src/ekf_node.cpp	Wrote real scan-to-scan 2D ICP with SVD-based rigid alignment; added gyro-bias subtraction in the IMU path; gated the ground-truth path behind a flag; documented every measurement model in-source.
src/ekf_estimator/config/ekf_params.yaml	Added imu_yaw_rate_bias, use_ground_truth, use_scan_matching, and ICP tuning parameters.
src/ekf_estimator/config/ekf_params_hardware.yaml	New file. Same shape as ekf_params.yaml but with use_ground_truth=false and noise values tuned for real sensors.
src/motion_planner/launch/hardware_demo.launch.py	New file. Brings up the EKF + nav stack WITHOUT Gazebo or the ros_gz bridge; expects real-hardware drivers running externally.

2. The estimator at a glance

2.1 State, prediction, covariance

The state is the planar pose plus body velocities. Each prediction step is a closed-form integration of the unicycle model with the current heading and twist held constant over dt . v and ω are modelled as Gaussian random walks: the mean does not evolve, but Q absorbs the uncertainty so v and ω can drift between measurement updates.

```

x_{k+1|k}(0) = x(0) + v cos(theta) dt
x_{k+1|k}(1) = x(1) + v sin(theta) dt
x_{k+1|k}(2) = wrap(theta + omega dt)
x_{k+1|k}(3) = v
x_{k+1|k}(4) = omega

```

```

P_{k+1|k} = F P F^T + Q dt

```

F is the Jacobian of the propagation w.r.t. the state, evaluated at the current x . The implementation (ekf_core.cpp) uses Joseph form for the covariance update so symmetry is preserved across Kalman gain rounding.

2.2 Measurement updates

Each sensor provides a partial measurement that is fused into the filter through its own linear H matrix and noise covariance R . There are three updates today, each implemented in ekf_core.cpp:

- updateImuYawRate(omega, R) - $z = \omega$ (scalar). H picks the omega state.

- `updateWheelOdom(z, R)` - $z = [v, \omega]$. H picks v and ω .
- `updatePose(z, R)` - $z = [x, y, \theta]$. H picks the pose states.

The heading innovation is wrapped to $[-\pi, \pi]$ before the update so we never apply a 6 rad correction when the true error is 0.28 rad.

3. Sensor mapping - sim to hardware

3.1 IMU - MPU6050

The MPU6050 is a 6-DoF IMU: a 3-axis gyroscope plus a 3-axis accelerometer, with no magnetometer. For a planar robot we use only the body-frame z-gyro (yaw rate). The accelerometer is unused in this filter because wheel encoders give a less-noisy linear velocity.

Message contract

```
topic : /imu
type  : sensor_msgs/Imu
fields used :
    angular_velocity.z      (rad/s)
everything else is ignored.
```

Any MPU6050 ROS 2 driver that publishes `sensor_msgs/Imu` drops in directly. Examples on Jazzy:

- `hytseng0509/mpu6050-ros2` (I2C, Raspberry Pi)
- the `linorobot2` hardware bring-up package
- any custom node using `smbus` / `pigpio` to drive the chip

Bias calibration (required)

MPU6050 gyros have a ~ 10 -50 mrad/s static bias that varies with temperature and supply voltage. Without subtraction, the EKF will spin its θ estimate at the bias rate whenever the robot is stationary. Calibrate after ~ 30 s of warm-up:

```
# 1. Leave the robot motionless and run for ~5 s.
ros2 topic echo /imu --field angular_velocity.z > /tmp/gz.log
# 2. Average the samples.
python3 -c "import statistics, sys; \
    vs=[float(l) for l in open('/tmp/gz.log') if l.strip() \
    and l.strip().replace('.', '').replace('-', '').replace('e', '').isdigit()]; \
    print(statistics.mean(vs))"
# 3. Put the NEGATED value into ekf_params_hardware.yaml as
#     imu_yaw_rate_bias.
```

Re-run after the robot has been on for several minutes; warm bias often differs by 1-3 mrad/s from cold bias.

Noise variance

The default `imu_yaw_rate_var = 3e-4 rad2/s2` is what an MPU6050 produces at the typical 100 Hz output rate, sitting on a vibrating chassis. Tighten if mounted on a damped platform, loosen if directly on a metal frame with motor noise.

3.2 LiDAR - RPLiDAR

Both the Gazebo `gpu_lidar` and the `rplidar_ros` driver publish the same `sensor_msgs/LaserScan` message, so the EKF callback is platform-agnostic. The scan-matching update implemented here is point-to-point 2D ICP between consecutive scans; the integrated delta gives a pose measurement.

Message contract

```
topic : /scan
type  : sensor_msgs/LaserScan
fields used :
    angle_min, angle_max, angle_increment
    range_min, range_max
    ranges[]
```

ICP algorithm

Each scan k:

- Convert to Cartesian: $(r_i \cdot \cos(a_i), r_i \cdot \sin(a_i))$. Drop non-finite, out-of-range readings.
- Sub-sample by scan_subsample (default 4) to keep brute-force NN tractable.
- ICP loop, up to scan_icp_max_iter (default 15): for every src point find the nearest dst point (squared distance below 0.25 m^2 to reject outliers), then run SVD-based rigid alignment on the paired set. Stop when $|\delta| < \text{scan_icp_eps}$.
- If the integrated XY jump exceeds scan_max_jump_m (0.5 m by default), the update is rejected as divergent.
- The accepted $(dx, dy, d\theta)$ is composed onto the EKF node's scan_pose accumulator and that absolute pose is fed to updatePose. R is scan_xy_var on the diagonal for XY and scan_yaw_var for theta.

Why scan-to-scan and not scan-to-map?

Scan-to-scan ICP is local and never diverges on a featureless patch the way an exhaustive scan-to-map search can. The trade-off is unbounded drift over long runs - the LiDAR estimate becomes another dead-reckoning source, not a global fix. We compensate by setting scan_xy_var an order of magnitude larger than the sim's ground-truth variance so the EKF does not over-trust it. For drift-bounded localisation, swap this for AMCL or slam_toolbox externally; the same EKF can then subscribe to AMCL's PoseWithCovarianceStamped instead.

RPLiDAR particulars

RPLiDAR A1 publishes 360 returns at 5.5-10 Hz; the A2/S1 publish 400-720 returns at 10-15 Hz. The scan_subsample default of 4 yields about 90-180 points per scan after dropping bad ranges, which keeps ICP under 5 ms on a Raspberry Pi 4. Increase scan_subsample on slower MCUs.

3.3 Wheel encoders

Message contract

```
topic : /odom
type  : nav_msgs/Odometry
fields used :
    twist.twist.linear.x      (m/s)
    twist.twist.angular.z     (rad/s)
POSE FIELD IS INTENTIONALLY IGNORED.
```

Real encoders measure wheel angular velocity. After the differential-drive forward kinematics that becomes (v, ω) . The nav_msgs/Odometry message also carries an integrated pose, but every controller that does the integration accumulates its own bias and slip. To avoid an unobservable double-integrator loop in the EKF, we deliberately ignore that pose and let the filter integrate v and ω itself, then correct with LiDAR / external pose.

Typical driver setup on hardware

- ros2_control with a diff_drive_controller plugin. Configure wheel_radius and wheel_separation to match the chassis.
- A custom node reading encoder ticks over UART/CAN; do the ticks-to-rad/s conversion in firmware where possible to minimise jitter.
- Either way, publish on /odom (or remap the EKF subscription if your driver uses a different topic).

4. Sim-only behaviour and how to disable it

4.1 Ground-truth pose subscription

Gazebo's PosePublisher emits one geometry_msgs/PoseStamped per link with frame_id = link name. In sim we use base_link's pose as a tight EKF correction to bound drift during demos. On hardware no such topic exists, so the node must not act on it.

Set use_ground_truth=false in ekf_params_hardware.yaml. The callback returns early and the subscription remains harmless even if nothing ever publishes the topic.

4.2 Initial pose

The sim's spawn is (-7, 7, 0) or (7, -7, 0) depending on the world; the hardware default is (0, 0, 0). Override per run via the initial_x / initial_y / initial_theta parameters or by sending an /initialpose message.

5. Parameter reference

Parameter	Default (sim)	Hardware default	Units / meaning
predict_rate_hz	50.0	50.0	Hz; EKF prediction timer rate.
q_x, q_y, q_theta	1e-4	1e-4	Process noise (per second) on pose.
q_v, q_omega	1e-2	2e-2	Random-walk noise on body twist.
imu_yaw_rate_var	1e-3	3e-4	(rad/s)^2; MPU6050 noise variance.
imu_yaw_rate_bias	0	tuned	rad/s; subtracted from gyro_z.
odom_v_var	1e-3	2e-3	(m/s)^2 on encoder linear.
odom_w_var	1e-3	2e-3	(rad/s)^2 on encoder angular.
scan_xy_var	5e-2	1e-1	(m)^2 on ICP XY.
scan_yaw_var	5e-2	1e-1	(rad)^2 on ICP heading.
use_ground_truth	true	false	Disable on hardware.
use_scan_matching	true	true	Disable if /scan is unusable.
scan_icp_max_iter	15	15	ICP iterations per scan.
scan_icp_eps	1e-4	1e-4	Convergence threshold on delta .
scan_subsample	4	4	Keep every Nth scan return.
scan_max_jump_m	0.5	0.5	m; outlier rejection.
initial_x / y / theta	sim spawn	0	World frame. Set via /initialpose if needed.
publish_tf	true	true	Broadcast odom -> base_link.

6. Hardware bring-up walkthrough

6.1 Wire up the sensors and confirm topics

```
# In separate terminals (or one launch file per driver):
ros2 run mpu6050_driver mpu6050_node          # -> /imu
ros2 launch rplidar_ros rplidar_a1_launch.py  # -> /scan
ros2 launch my_robot diff_drive.launch.py     # -> /odom, accepts /cmd_vel
```

```
# Verify the topics are alive and the message types match:
ros2 topic hz /imu
ros2 topic hz /scan
ros2 topic hz /odom
ros2 topic echo /imu --once
ros2 topic echo /scan --once | head -8
ros2 topic echo /odom --once | head -20
```

6.2 Calibrate the IMU bias

With the robot powered up, warm for ~30 s, then leave it perfectly still on the ground. Use the snippet from section 3.1 to compute the mean of gyro_z over 5 s and write the negated value into `ekf_params hardware.yaml`.

6.3 Tune encoder noise if needed

Drive the robot at a constant 0.2 m/s for ~10 s. Compute `stddev(twist.linear.x)` over that window from `/odom`. Square it and slot the result into `odom_v_var`.

6.4 Launch the EKF + navigation

```
ros2 launch motion_planner hardware_demo.launch.py

# Optional: pick a different planner.
ros2 launch motion_planner hardware_demo.launch.py planner_type:=rrt_star
```

6.5 Send a goal from RViz or the CLI

```
ros2 topic pub --once /goal_pose geometry_msgs/PoseStamped \
  "{header: {frame_id: 'odom'}, pose: \
    {position: {x: 2.0, y: 0.0}, orientation: {w: 1.0}}}"
```

6.6 Common failure modes

- Theta drifts when stationary -> gyro bias not subtracted. Re-do step 6.2.
- EKF pose jumps every scan -> ICP is diverging on a sparse feature set. Raise `scan_subsample`, lower `scan_max_jump_m`, or set `use_scan_matching=false` and rely on wheel + IMU.
- Robot drives in tight circles after a goal -> Pure Pursuit tracking bug. Make sure the `navigation_node` binary is from the post-fix build (closest-waypoint anchored lookahead).
- TF tree breaks (rviz: 'no transform from odom to base_link') -> `publish_tf` is false. Either set it to true here or have some other node publish that transform.

7. Math reference

7.1 Why bias subtraction is a measurement-level fix

If the gyro has a true bias b that we do not model, the innovation $z - h(x)$ on every IMU update is shifted by b . Over time the filter compensates by walking ω (and through it, θ) along with the bias. Two cures: (a) subtract a calibrated constant before the update (what we do), or (b) augment the state with a bias term and let the filter estimate it online. The augmented-state approach needs an additional Q on the bias and a measurement that constrains θ absolutely - on this platform that constraint comes from the LiDAR pose update, and you do want the bias to be observable. We left it out to keep the filter at 5 states for the demo; promoting to 7 (add `bias_omega` and `bias_v` if you want a wheel-slip term) is a strict superset.

7.2 SVD-based 2D rigid alignment

Given paired point sets $\{s_i\}$ and $\{d_i\}$, the optimal rigid transform (R, t) minimising $\sum |R s_i + t - d_i|^2$ is:

```
mu_s = mean(s_i),    mu_d = mean(d_i)
W     = sum (d_i - mu_d)(s_i - mu_s)^T
USV^T = svd(W)
R     = U diag(1, det(UV^T)) V^T    # forces a proper rotation
t     = mu_d - R mu_s
dtheta = atan2(R(1,0), R(0,0))
```

This is the planar specialisation of Arun's 1987 method (see Kabsch/Umeyama). The $\text{diag}(1, \det)$ factor flips one axis if the SVD returns a reflection - critical when scan-to-scan motion is near zero.

7.3 Why ignore the encoder pose field

`nav_msgs/Odometry` carries both twist and pose; the pose is an integration of twist with whatever bias the controller accumulates. If the EKF were to ingest both, we would feed the same noise back into the filter twice with two different weights, and the resulting estimate would be biased even in steady state. Worse, our state already integrates v and ω , so taking the encoder pose would be a third path - the ML cost of unobservable redundancy. Pure velocity in, EKF does the integration, LiDAR / external pose corrects.

8. Testing the new measurement models

8.1 Unit tests (existing)

`ekf_estimator` already ships GTest unit tests against `EkfCore` (`test_ekf_core.cpp`). They cover `predict()` against an analytical reference and the three update methods against their linear H formulations. The new scan-matching code is static-method-shaped so it can be tested in isolation; the helpers exposed in the header are `scanToPoints`, `rigidAlign2D`, and `icp2d`.

8.2 In-sim verification

```
ros2 launch motion_planner nav_demo.launch.py launch_rviz:=false
ros2 topic echo /ekf/pose --once      # sanity
ros2 topic pub --once /goal_pose geometry_msgs/PoseStamped \
  "{header: {frame_id: 'odom'}, pose: \
    {position: {x: 2.0, y: 0.0}, orientation: {w: 1.0}}}"
```

8.3 Sim -> hardware checklist

- Set `use_ground_truth=false`.
- Set `imu_yaw_rate_bias` from the calibration step.
- Confirm `/imu`, `/odom`, `/scan` are all flowing at expected rates.
- Confirm the diff-drive driver subscribes to `/cmd_vel` and the wheels respond.
- Confirm the TF tree contains `odom -> base_link` (either from this EKF or from your bring-up).
- Send a small goal (1 m forward) and confirm the robot actually moves AND the `/ekf/pose` advances accordingly.

Appendix A - Topic and parameter summary

Topic	Type	Direction	Used for
/imu	sensor_msgs/Imu	in	yaw-rate update (gyro_z)
/odom	nav_msgs/Odometry	in	twist (v, omega) update
/scan	sensor_msgs/LaserScan	in	scan-to-scan ICP
ground_truth/pose	geometry_msgs/PoseStamped	in	sim only; gated by use_ground_truth
/ekf/pose	geometry_msgs/PoseWithCovarianceStamped	out	fused estimate
tf (odom -> base_link)	tf2_msgs/TFMessage	out	broadcast if publish_tf=true